Architectural Divergence in Agentic Software Engineering (2024–2026): A Comparative Analysis of Google Gemini Conductor and the Emerging "nukeCE" Ecosystem

1. Introduction: The Agentic Phase Transition

The trajectory of software engineering in the mid-2020s has been defined by a decisive shift from "stochastic code generation" to "stateful agentic orchestration." The period between 2024 and 2026 witnessed the obsolescence of the chat-based copilot model—characterized by ephemeral context windows and stateless interactions—in favor of architectures that embed artificial intelligence directly into the persistent lifecycle of the repository. This report provides an exhaustive comparative analysis of this transition, focusing specifically on the architectural conflict between **Google Gemini Conductor** (representing the "Context-Driven Development" paradigm) and the broader ecosystem of autonomous agents, including **SWE-agent**, **Claude Code**, and **Devin**, colloquially and collectively referred to in advanced development circles as the "nukeCE" (Next-Generation Unified Knowledge & Execution Context Environment) class of systems.

The core thesis of this analysis posits that the differentiation between these systems is no longer a function of the underlying Large Language Model (LLM) intelligence. Instead, the competitive advantage has migrated to the **Agent-Computer Interface (ACI)** and the mechanisms of **State Persistence**. As systems move from suggesting code to architecting features, the ability to maintain a coherent "reality" for the agent—and to persist that reality across discontinuous sessions—has become the primary determinant of engineering velocity and safety.

This report examines three critical dimensions of this ecosystem:

1. **State Persistence Architectures:** The divergence between file-based state (Conductor), context-cached state (Claude), and trajectory-based state (SWE-agent).
2. **Interface Reality Modeling:** How ACIs construct a simplified "reality" to prevent agent cognitive overload, analyzing the theoretical underpinnings of "interface reality" in the context of 2026 AI workflows.
3. **Deterministic Governance:** The rise of "Agentic Firewalls" and deterministic guardrails necessary to deploy probabilistic agents in enterprise environments.

The analysis draws upon technical documentation, architectural whitepapers, and rigorous benchmarks from the 2024–2026 period.[1]

---

2. The Google Gemini Conductor Architecture: Context as Code

Google Gemini Conductor, introduced in late 2025, represents a fundamental re-architecture of the AI developer workflow. Unlike its predecessors, which treated context as a transient property of a chat window, Conductor treats context as a **managed artifact** within the version control system. This approach, termed **Context-Driven Development (CDD)**, seeks to solve the "drift" problem inherent in long-context LLM interactions.[1]

2.1 The Philosophy of Repository-Resident State

The central innovation of Conductor is the externalization of agent cognition into persistent Markdown files (plan.md, spec.md, status.md). In traditional workflows, an agent's "understanding" of a task exists solely in the KV-cache of the inference engine. If the session terminates, the understanding evaporates. Conductor inverts this: the agent's understanding is anchored in the repository itself.

Table 1: The Artifact Hierarchy in Google Gemini Conductor [1]

Artifact	Location	Function	Persistence Model	Cognitive Role
Product Context	conductor/product.md	Defines high-level product goals, user personas, and visionary constraints.	**Global / Static**: Rarely changes; serves as the "constitution" for the agent.	Long-Term Semantic Memory
Tech Stack	conductor/tech-stack.md	Explicit definition of languages, frameworks, and patterns (e.g., "FastAPI backend").	**Global / Static**: Ensures consistent architectural decisions.	Constraint Boundary

Workflow Rules	conductor/workflow.md	Defines process constraints (e.g., "Must use TDD", "Verify via User Manual").	**Global / Procedural**: Dictates the *behavior* of the agent.	Procedural Memory / Superego
The Track	conductor/tracks/	A container for a specific unit of work (feature or bug fix).	**Local / Dynamic**: Created for a specific task; archived upon completion.	Episodic Memory
The Plan	tracks/.../plan.md	A mutable, phased to-do list that tracks execution state ([ ] to [x]).	**Local / Mutable**: The "working memory," updated in real-time.	Executive Function

This architecture transforms the repository into the "Single Source of Truth." When an agent pauses or a human developer switches machines, the state is preserved not in a vendor's proprietary cloud database, but in the Git history of the repository itself. This enables **asynchronous collaboration** between human and machine; a human can edit the plan.md to correct the agent's course without needing to "prompt engineer" the correction.[6]

2.2 The plan.md as an Executable State Machine

The most critical component of Conductor is the plan.md file. It acts as an **executable state machine**. When the agent runs, it parses this file to determine its current state.

- **Initialization:** The command /conductor:newTrack triggers the generation of a spec.md (Requirements) and a plan.md (Execution Strategy).[1]
- **Execution Loop:** During /conductor:implement, the agent reads the plan, identifies the next unchecked task, executes it, and then *modifies the file* to mark it as complete ([x]).
- **Drift Mitigation:** Because the agent must physically write its progress to the file, "hallucinated progress" is minimized. If the agent thinks it has completed a task but fails to check the box, the system forces a re-evaluation.[5]

2.2.1 The Checkpointing Protocol and Verification

Conductor enforces a strict "Checkpointing Protocol." The workflow.md file allows teams to define mandatory gates, such as a "User Manual Verification" step at the end of each phase.[5] This integration of "Human-in-the-Loop" (HITL) workflows prevents the "cascading error" phenomenon observed in autonomous agents like Devin, where a small initial error compounds into a completely divergent codebase.[8]

By centralizing technical constraints (e.g., "Always use Pytest"), Conductor ensures that every AI-generated contribution adheres to the team's specific guidelines, regardless of which developer initiates the session. This solves the "Junior Developer on Espresso" problem [5], where unconstrained agents produce working but non-idiomatic code.

2.3 Integration with "Antigravity" and Long-Term Retention

Recent discussions in the ecosystem suggest that Conductor's file-based approach is being integrated into broader Google internal systems like "Antigravity".[9] This integration points to a future where "Long-term Product Context" is not just a Markdown file but a dynamic Workspace Rule (e.g., project-goals.md inside .agent/rules/). This suggests a convergence where the *file system* becomes the *database* for the agent's long-term memory, leveraging the version control capabilities of Git to manage the evolution of the agent's understanding over time.

---

3. Comparative Analysis: Conductor vs. The "nukeCE" Ecosystem

While Conductor relies on explicit, user-visible Markdown files, the broader "nukeCE" ecosystem—comprising **SWE-agent**, **Claude Code (MCP)**, and **Devin**—employs divergent architectures for state persistence and reality modeling. This section contrasts these approaches.

3.1 SWE-agent: The Trajectory-Based Epistemology

SWE-agent, developed by researchers at Princeton and Stanford, represents a "Trajectory-Based" approach to state.[2] Unlike Conductor's human-readable plan, SWE-agent manages state through **Trajectory Logs** (*.traj).

3.1.1 The Role of Trajectories

A "trajectory" captures the complete sequence of the agent's "Thought," "Command," and "Observation."

- **Implicit vs. Explicit State:** Conductor uses *explicit* state (a checklist). SWE-agent uses *implicit* state (the history of actions). The agent must infer its progress by analyzing its

own past trajectory.[11]

- **Self-Evolution (SE-Agent):** Advanced iterations of SWE-agent employ "Self-Evolution" frameworks. By analyzing past trajectories (Revision, Recombination, Refinement), the agent optimizes its reasoning process for future tasks.[2] This allows the agent to "learn" from its mistakes without updating a static rule file like workflow.md.

Insight: SWE-agent's approach is superior for *research and autonomous debugging*, where the path to the solution is unknown. Conductor's approach is superior for *feature development*, where the path can be planned in advance.

3.2 Claude Code & MCP: The Tool-Mediated Reality

Anthropic's **Claude Code**, empowered by the **Model Context Protocol (MCP)**, represents a "Tool-Mediated" architecture.[12]

3.2.1 Context Caching and Programmatic Tool Calling

Claude Code addresses the state persistence problem not by writing files, but by optimizing the **Context Window**.

- **Programmatic Tool Calling (PTC):** In standard workflows, tool outputs (e.g., a 5,000-line database dump) flood the context window, causing the model to "forget" earlier instructions. Claude's PTC allows the agent to write an orchestration script (e.g., in Python) that processes this data in a sandbox and returns *only the final result* to the context.[13]
- **Drift Management:** By keeping intermediate results out of the context, Claude Code preserves the "signal-to-noise" ratio, effectively maintaining state longer than standard chat models.

3.2.2 The CLAUDE.md Anchor

Similar to Conductor's product.md, Claude Code relies on a CLAUDE.md file for project context.[14] However, unlike Conductor's plan.md, CLAUDE.md is typically **Read-Only**. The community has spontaneously adopted "Plan Mode" workflows—manually creating plan.md files—to bridge this gap.[7] This convergence proves that **Repo-Resident State** is the dominant design pattern for 2026.

3.3 Devin and OpenDevin: The Sandboxed Reality

Devin (and its open-source counterpart **OpenDevin/OpenHands**) operates in a fully **Sandboxed Cloud Environment**.[16]

- **State as a Virtual Machine:** Devin's state is the entire state of the Virtual Machine (VM). If Devin installs a dependency, it persists in the VM. This differs from Conductor, where state is text in a file.
- **Opacity vs. Transparency:** Devin's internal state (its "thoughts" and "plans") is often

opaque or presented via a proprietary UI widget.[8] Conductor's state is transparent Markdown. If Devin gets stuck, the user must "prompt" it. If Conductor gets stuck, the user can "edit" the plan.

Table 2: Comparative Analysis of State Persistence Architectures (2024–2026)

Feature	Google Gemini Conductor	SWE-agent / SE-Agent	Claude Code (MCP)	Devin / OpenDevin
Primary State Artifact	plan.md (Markdown Checklist)	*.traj (Trajectory Log)	Context Window / CLAUDE.md	Cloud VM Snapshot
State Mutability	**Bi-directional**: Agent and User both edit the plan.	**Append-Only**: Agent generates trajectory; analyzed post-hoc.	**Read-Only Anchor**: CLAUDE.md is static; context is fluid.	**Opaque**: State is managed internally by the SaaS.
Drift Mitigation	**Explicit Planning**: Agent realigns against the plan.md.	**Trajectory Pruning**: History is summarized to fit context.	**Context Optimization**: PTC hides intermediate data.[13]	**Unified UI**: Sidebar plan widget maintains focus.
Session Portability	**Git-Based**: Works across any machine with the repo.[1]	**Replay**: Can replay trajectories, but tied to runtime.	**Teleportation**: claude --teleport moves session state.[18]	**SaaS-Bound**: Resides in the vendor's cloud.
Cognitive Architecture	**Planner-Executor**: Separates planning (plan.md) from coding.	**ReAct Loop**: Iterative Thought-Action-Observation.	**Orchestrator**: Writes code to manage tools (PTC).	**Autonomous Agent**: Full-stack autonomous execution.

4. Theoretical Framework: Interface Reality and ACI Design

The second critical axis of comparison is **Interface Reality**—the constructed environment the agent perceives. The research literature [3] distinguishes between "Reality" (the raw OS/Codebase) and the "Interface" (how it is presented). For AI agents, the **Agent-Computer Interface (ACI)** determines the "Physics" of their reality.

4.1 The Theory of Constructed Reality

Theoretical frameworks on "Interface Reality" posit that an interface "changes what and how we see, how we experience reality and how this reality is configured".[3] In the context of "nukeCE" systems, the ACI creates a **Simulacrum of Development**.

- **The Universe:** The codebase and OS.
- **The Application:** The LLM's cognitive core.
- **The Interface (ACI):** The filter that translates the Universe into tokens.[19]

4.2 SWE-agent: The Constraint-Based Reality

SWE-agent explicitly engineers this reality to overcome LLM limitations.

- **Visual Constraints:** The open command in SWE-agent restricts the view to **100 lines** at a time.[20] The agent effectively exists in a reality where files are viewed through a "keyhole." This prevents the "Lost in the Middle" phenomenon.
- **Linting as Physics:** The ACI includes a "linter guardrail" on the edit command. If the agent submits invalid syntax, the ACI rejects the command.[20] In the agent's constructed reality, **it is physically impossible to save broken code**. This "Physics of Correctness" is a profound architectural decision that separates SWE-agent from raw shell tools.

4.3 Claude Code & MCP: The API-Based Reality

Claude Code uses MCP to construct a **Tool-Based Reality**.

- **Abstraction:** Instead of seeing a raw file system, the agent sees a list of Tools (read_file, git_commit).[21] The "Universe" is abstracted into a set of API calls.
- **Dynamic Construction:** Through **Tool Discovery**, the agent builds its reality on the fly. It does not know a tool exists until it searches for it.[13] This models reality as a "Fog of War" that is revealed through exploration, contrasting with Conductor's "Global Context" (tech-stack.md) which presents the reality upfront.

4.4 GazeCopilot and Biometric Reality

Emerging research into **Real-time GazeCopilot** [22] suggests a future where "Interface Reality" incorporates human biometric data. By tracking where a developer looks (fixation patterns), the system infers cognitive state and adjusts the prompt context. While not yet standard in Conductor or SWE-agent, this represents the next frontier: an interface reality that is **shared** between human and agent, rather than separate.

---

5. State Persistence and the Problem of "Drift"

A recurring failure mode in agentic workflows is **Drift**: the gradual divergence of the agent's actions from the user's intent.[23]

5.1 The Mechanics of Drift

Drift occurs due to the non-deterministic nature of LLMs and the accumulation of "noise" in the context window.

- **Silent Failure:** The agent selects the wrong tool or hallucinates a requirement.[24]
- **Cycles:** The agent gets stuck in a loop of planning and replanning.
- **Missing Details:** The final output is syntactically correct but functionally incomplete.

5.2 Conductor's Solution: The Anchored Plan

Conductor mitigates drift through **File-Based Anchoring**. Because the plan.md is persistent and external to the context window, it acts as a "checksum" for the agent's behavior. Before every action, the agent must reconcile its intent with the plan.md. If the plan says "Implement Auth," and the agent tries to "Refactor CSS," the mismatch is detected (either by the agent's self-reflection or the user's review of the file).[6]

5.3 Deterministic Drift Detection

Enterprise-grade systems are now employing **Drift Detection** tools (e.g., from IBM watsonx or Amazon Bedrock).[23] These systems use "Prompt-Driven LLM Agents" to monitor the infrastructure or codebase for deviations from the "Desired State" defined in the prompts.[25] In the context of software development, Conductor's spec.md serves as the "Desired State" against which the code is measured.

---

6. Deterministic Guardrails and the "Agentic Firewall"

The transition from "Chat" to "Action" (e.g., executing code, modifying databases) necessitates a shift from **Probabilistic Safety** (RLHF) to **Deterministic Safety** (Firewalls).

6.1 The "rm -rf" Problem

An autonomous agent with shell access poses a catastrophic risk. An innocuous command like "clean up temp files" could be misinterpreted as rm -rf /.[4] Probabilistic refusals ("I'm sorry, I can't do that") are insufficient because they can be "jailbroken" or bypassed by context confusion.[26]

6.2 The Agentic Firewall Architecture

To solve this, architectures like **LangChain Middleware** and **AgentGateway** [27] are being deployed as "Agentic Firewalls".[4]

Table 3: Layers of Agentic Defense [4]

Layer	Mechanism	Deterministic/Probabilistic	Example
Input	**Sanitization Agents**	Probabilistic	Detecting prompt injection attacks (e.g., "Ignore previous instructions").
Middleware	**Regex / Allowlisting**	**Deterministic**	Blocking rm -rf, chmod 777, or DROP TABLE via regex before execution.
RBAC	**Identity Management**	**Deterministic**	Running the agent as a user with read-only access to /etc and write access only to /src.
Output	**PII Filtering**	Hybrid	Redacting API keys or credit card numbers from logs.[30]

Logic	**Rule Engines (DMN)**	**Deterministic**	"If confidence < 0.9, require Human Approval".[31]

6.3 Conductor's Approach: Workflow-Mediated Safety

Conductor implements safety through **Workflow-Mediated Governance**.

- **The Checkpoint:** By mandating a "User Manual Verification" at the end of a phase [5], Conductor ensures that the blast radius of an agent's error is limited to a single phase.
- **Smart Revert:** Conductor's /conductor:revert is git-aware. It allows the user to undo an entire "Track" or "Phase" instantly, providing a "Time Machine" to recover from agentic errors.[5]

6.4 Hard Constraints vs. Safe Completions

There is an ongoing architectural debate between **Hard Refusals** (blocking the action) and **Safe Completions** (sanitizing the action).

- **Hard Refusals:** Standard in "Guardrails AI".[29] If the agent tries to access a restricted file, the firewall returns an error.
- **Safe Completions:** Proposed in newer research [26], where the model is trained to *comply safely* (e.g., providing a summarized version of the file instead of the raw data) rather than refusing. Conductor tends towards this by using the tech-stack.md to guide the agent toward safe, approved patterns rather than just blocking bad ones.

---

7. The Human Interface: Merge Conflicts and Collaboration

A unique challenge of Conductor's file-based state is the reintroduction of **Merge Conflicts**.[32] If both the human and the agent edit the plan.md simultaneously, or if multiple agents working on different "Tracks" modify the global tracks.md, conflicts arise.

7.1 Managing the Agentic Merge

Traditional git workflows assume human-to-human conflict resolution. With agents:

- **Agent-Aware Resolution:** Tools like **Graphite** and **JetBrains AI Assistant** are evolving to resolve these conflicts automatically by understanding the *semantic intent* of the changes.[33]
- **Conductor's Locking Mechanism:** Conductor manages "Tracks" in separate subdirectories (conductor/tracks/TRK-001/), minimizing collision risk on the state files themselves. However, collisions in the *source code* remain a challenge.

- **Frequent Integration:** The best practice for agentic workflows is "Frequent Integration".[32] Because agents work fast, the divergence between the main branch and the agent's branch grows exponentially. Conductor's "Phase" structure encourages merging after every verified phase, keeping this divergence manageable.

---

8. Conclusion: The Convergence of nukeCE Architectures

The comparative analysis of Google Gemini Conductor against the "nukeCE" ecosystem (SWE-agent, Claude Code, Devin) reveals a definitive convergence toward **Stateful, Repo-Resident Architectures**.

8.1 Key Findings

1. **Persistence Wins:** The "Chat" model is dead. The "File" model (Conductor's plan.md) and the "Trajectory" model (SWE-agent's .traj) are the only viable paths for long-duration complex tasks. Conductor's approach is more human-collaborative, while SWE-agent's is more machine-optimizable.
2. **Reality is Constructed:** High-performance agents do not look at raw code. They look at **ACIs** (100-line windows, tool abstractions). The success of SWE-agent proves that *limiting* the agent's reality is key to focusing its intelligence.
3. **Safety is Deterministic:** You cannot prompt-engineer safety. Real-world deployment requires **Agentic Firewalls** that deterministically block dangerous syscalls and enforce RBAC.

8.2 Future Outlook (2026)

As we look toward late 2026, we anticipate the emergence of **Hybrid Systems** that combine Conductor's planning with SWE-agent's execution.

- **The "Antigravity" Synthesis:** Google's internal "Antigravity" project [9] likely represents the fusion of Conductor's high-level context management with a DeepMind-driven, trajectory-optimizing execution engine.
- **Standardization of ACI:** The **Model Context Protocol (MCP)** [35] is poised to become the standard ACI, allowing any agent (Conductor, Claude, or Devin) to plug into any reality (Local, Cloud, Database).

In this rapidly evolving landscape, **Google Gemini Conductor** stands out as the most pragmatic architecture for *augmented* human development, offering a "Glass Box" approach where state is transparent and mutable. Conversely, **SWE-agent** and **Devin** represent the path toward *autonomous* development, operating as "Black Boxes" that optimize for outcomes rather than collaboration. The choice between them depends ultimately on whether

the organization views AI as a **Tool** (Conductor) or a **Workforce** (Devin).

Works cited

1. Conductor: Introducing context-driven development for Gemini CLI ..., accessed January 14, 2026, https://developers.googleblog.com/conductor-introducing-context-driven-development-for-gemini-cli/
2. Self-Evolution Trajectory Optimization in Multi-Step Reasoning with LLM-Based Agents, accessed January 14, 2026, https://arxiv.org/html/2508.02085v6
3. curating web-based art exhibitions: mapping their migration and integration with offline formats of product - CORE, accessed January 14, 2026, https://core.ac.uk/download/pdf/74368667.pdf
4. Agentic AI: Production-Ready Security Blueprint - AI, Robotics, Ops, accessed January 14, 2026, https://alexostrovskyy.com/the-agentic-firewall-a-production-ready-security-blueprint-for-your-agentic-ai/
5. Trying Out the New Conductor Extension in Gemini CLI — We're ..., accessed January 14, 2026, https://medium.com/google-cloud/trying-out-the-new-conductor-extension-in-gemini-cli-0801f892e2db
6. Google Introduces Conductor, a Context-Driven Development Extension for Gemini CLI, accessed January 14, 2026, https://www.infoq.com/news/2026/01/google-conductor/
7. When to use Plan Mode vs Markdown Documentation : r/ClaudeAI - Reddit, accessed January 14, 2026, https://www.reddit.com/r/ClaudeAI/comments/1nrejut/when_to_use_plan_mode_vs_markdown_documentation/
8. Devin Docs - Devin AI, accessed January 14, 2026, https://docs.devin.ai/
9. Conductor should be integrated into Antigravity to ensure long-term Context retention, accessed January 14, 2026, https://discuss.ai.google.dev/t/conductor-should-be-integrated-into-antigravity-to-ensure-long-term-context-retention/113384
10. SWE-agent: An In-depth Analysis of Core Concepts, Performance, Real-World Applications, and Future Outlook - MGX, accessed January 14, 2026, https://mgx.dev/insights/swe-agent-an-in-depth-analysis-of-core-concepts-performance-real-world-applications-and-future-outlook/01411af5633b4d5e99df74fe14959586
11. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering - arXiv, accessed January 14, 2026, https://arxiv.org/pdf/2405.15793?
12. Claude's Model Context Protocol: A Deep Dive into Server Intercommunication, Buffer Memory, and RAG..., accessed January 14, 2026, https://nehalmr.medium.com/claudes-model-context-protocol-a-deep-dive-into-server-intercommunication-buffer-memory-and-rag-ee454c46b51b
13. Introducing advanced tool use on the Claude Developer Platform - Anthropic,

accessed January 14, 2026, https://www.anthropic.com/engineering/advanced-tool-use
14. A week with Claude Code: lessons, surprises and smarter workflows - DEV Community, accessed January 14, 2026, https://dev.to/ujjavala/a-week-with-claude-code-lessons-surprises-and-smarter-workflows-23ip
15. Mastering the Vibe: Claude Code Best Practices That Actually Work - Dinanjana Gunaratne, accessed January 14, 2026, https://dinanjana.medium.com/mastering-the-vibe-claude-code-best-practices-that-actually-work-823371daf64c
16. Rapid web app development with Devin - A Developer's Perspective - Scott Logic Blog, accessed January 14, 2026, https://blog.scottlogic.com/2025/10/20/rapid-development-with-devin.html
17. OpenHands: An Open Platform for AI Software Developers as Generalist Agents - arXiv, accessed January 14, 2026, https://arxiv.org/html/2407.16741v3
18. Claude Code on the web, accessed January 14, 2026, https://code.claude.com/docs/en/claude-code-on-the-web
19. Automatic Generationof Interfaces using Constraints - OHSU's Digital Collections, accessed January 14, 2026, https://digitalcollections.ohsu.edu/record/241/files/241_etd.pdf
20. SWE-agent/docs/background/aci.md at main - GitHub, accessed January 14, 2026, https://github.com/SWE-agent/SWE-agent/blob/main/docs/background/aci.md
21. Model Context Protocol (MCP). MCP is an open protocol that... | by Aserdargun, accessed January 14, 2026, https://medium.com/@aserdargun/model-context-protocol-mcp-e453b47cf254
22. GazeCopilot: Evaluating Novel Gaze-Informed Prompting for AI-Supported Code Comprehension and Readability - arXiv, accessed January 14, 2026, https://arxiv.org/pdf/2511.08177
23. AWS Marketplace: Tailwinds OnRamp - LLM Drift Detection - Amazon.com, accessed January 14, 2026, https://aws.amazon.com/marketplace/pp/prodview-ssvzq4catjsrk
24. Detecting Silent Failures in Multi-Agentic AI Trajectories - arXiv, accessed January 14, 2026, https://arxiv.org/html/2511.04032v1
25. (PDF) Maintaining Infrastructure Drift with Prompt-Driven LLM Agents - ResearchGate, accessed January 14, 2026, https://www.researchgate.net/publication/399620082_Maintaining_Infrastructure_Drift_with_Prompt-Driven_LLM_Agents
26. (PDF) From Hard Refusals to Safe-Completions: Toward Output-Centric Safety Training, accessed January 14, 2026, https://www.researchgate.net/publication/395809333_From_Hard_Refusals_to_Safe-Completions_Toward_Output-Centric_Safety_Training
27. agentgateway/agentgateway: Next Generation Agentic Proxy for AI Agents and MCP servers - GitHub, accessed January 14, 2026, https://github.com/agentgateway/agentgateway

28. AI Manipulation Defense System - GitHub Gist, accessed January 14, 2026, https://gist.github.com/ruvnet/4cc23f3d3a97a0d8acd80693407b9a67
29. Guardrails for AI Agents - UX Planet, accessed January 14, 2026, https://uxplanet.org/guardrails-for-ai-agents-24349b93caeb
30. Essential AI agent guardrails for safe and ethical implementation - Toloka AI, accessed January 14, 2026, https://toloka.ai/blog/essential-ai-agent-guardrails-for-safe-and-ethical-implementation/
31. AI agent or Rule-based DMN? Choosing the Right Abstraction in AI-Powered Orchestration, accessed January 14, 2026, https://camunda.com/blog/2025/07/ai-agent-or-based-rule-dmn-ai-powered-orchestration/
32. How to Resolve Merge Conflicts in Git: Methods and Practices - Flosum, accessed January 14, 2026, https://www.flosum.com/blog/how-to-resolve-merge-conflicts-in-git
33. The role of AI in merge conflict resolution - Graphite, accessed January 14, 2026, https://graphite.com/guides/ai-code-merge-conflict-resolution
34. Resolve Git Merge Conflicts faster with Artificial Intelligence (AI) - DISCOVER, accessed January 14, 2026, https://www.arcadsoftware.com/discover/resources/blog/resolve-git-merge-conflicts-faster-with-artificial-intelligence-ai/
35. Model Context Protocol, accessed January 14, 2026, https://modelcontextprotocol.io/